

# Assignment-15.5

**Ch.Sushanth**  
**2303A51750**  
**Batch-11**

## Task 1 – Student Attendance API

**Task:**

**Create a RESTful API for student attendance tracking.**

**Instructions:**

- **Endpoints required:**
  - o GET /attendance → View attendance records
  - o POST /attendance → Mark attendance
  - o PUT /attendance/{id} → Update attendance record
  - o DELETE /attendance/{id} → Remove attendance entry
- **Ensure JSON-based request and response handling.**

**Expected Output:**

- **API for maintaining and updating attendance records.**

### PROMPT:

"""Generate a Python Flask REST API for student attendance with in-memory storage.

Implement GET /attendance, POST /attendance, PUT /attendance/<id>, DELETE /attendance/<id>.

Use JSON request/response and add comments."""

**CODE:**

```

lab15.py > create_attendance
1  """Generate a Python Flask REST API for student attendance with in-memory storage.
2  Implement GET /attendance, POST /attendance/<id>, DELETE /attendance/<id>.
3  Use JSON request/response and add comments."""
4  from flask import Flask, request, jsonify
5  app = Flask(__name__)
6  # In-memory storage for attendance records
7  attendance_records = []
8  # GET /attendance - Retrieve all attendance records
9  @app.route('/attendance', methods=['GET'])
10 def get_attendance():
11     return jsonify(attendance_records), 200
12 # POST /attendance - Create a new attendance record
13 @app.route('/attendance', methods=['POST'])
14 def create_attendance():
15     data = request.get_json()
16     if not data or 'student_name' not in data or 'date' not in data:
17         return jsonify({'error': 'Invalid input'}), 400
18     new_record = {}
19     'id': len(attendance_records) + 1,
20     'student_name': data['student_name'],
21     'date': data['date']
22 }
23 attendance_records.append(new_record)
24 return jsonify(new_record), 201
25 # PUT /attendance/<id> - Update an existing attendance record
26 @app.route('/attendance/<int:record_id>', methods=['PUT'])
27 def update_attendance(record_id):
28     data = request.get_json()
29     if not data or 'student_name' not in data or 'date' not in data:
30         return jsonify({'error': 'Invalid input'}), 400
31     for record in attendance_records:
32         if record['id'] == record_id:
33             record['student_name'] = data['student_name']
34             record['date'] = data['date']
35             return jsonify(record), 200
36     return jsonify({'error': 'Record not found'}), 404
37 # DELETE /attendance/<id> - Delete an attendance record
38 @app.route('/attendance/<int:record_id>', methods=['DELETE'])
39 def delete_attendance(record_id):
40     global attendance_records
41     attendance_records = [record for record in attendance_records if record['id'] != record_id]
42     return jsonify({'message': 'Record deleted'}), 200
43 if __name__ == '__main__':
44     app.run(debug=True)
45

```

#### Explanation:

This task creates a Flask API to manage student attendance records. It allows users to view, add, update, and delete attendance data using REST endpoints. The data is stored in memory and handled using JSON format.

## Task 2 – Inventory Management API

### Task:

Develop a REST API for warehouse inventory management.

### Instructions:

- Endpoints required:
  - o GET /inventory → List all items
  - o POST /inventory → Add new item
  - o PUT /inventory/{id} → Update stock quantity
  - o DELETE /inventory/{id} → Remove an item

- Implement validation for stock values.

### Expected Output:

- API capable of managing inventory stock levels.

### PROMPT:

"""Create a Flask REST API for inventory management with item id, name, and quantity.

Implement GET, POST, PUT, DELETE endpoints for /inventory.

Add validation so stock quantity cannot be negative."""

### Code:

```
1 """Create a Flask REST API for inventory management with item id, name, and quantity.
2 Implement GET, POST, PUT, DELETE endpoints for /inventory.
3 Add validation so stock quantity cannot be negative."""
4 from flask import Flask, request, jsonify
5 app = Flask(__name__)
6 # In-memory storage for inventory items
7 inventory = {}
8 @app.route('/inventory', methods=['GET'])
9 def get_inventory():
10     """Endpoint to retrieve the entire inventory."""
11     return jsonify(inventory)
12 @app.route('/inventory', methods=['POST'])
13 def add_item():
14     """Endpoint to add a new item to the inventory."""
15     data = request.get_json()
16     item_id = data.get('id')
17     name = data.get('name')
18     quantity = data.get('quantity')
19
20     # Validate input
21     if not isinstance(item_id, int) or not isinstance(name, str) or not isinstance(quantity, int):
22         return jsonify({"error": "Invalid input: id must be an integer, name must be a string, and quantity must be an integer."}), 400
23     if quantity < 0:
24         return jsonify({"error": "Invalid input: quantity cannot be negative."}), 400
25
26     # Add item to inventory
27     inventory[item_id] = {"name": name, "quantity": quantity}
28     return jsonify({"message": "Item added successfully."}), 201
29 @app.route('/inventory/<int:item_id>', methods=['PUT'])
30 def update_item(item_id):
31     """Endpoint to update an existing item in the inventory."""
32     if item_id not in inventory:
33         return jsonify({"error": "Item not found."}), 404
34
35     data = request.get_json()
36     name = data.get('name')
37     quantity = data.get('quantity')
38
39     # Validate input
40     if name is not None and not isinstance(name, str):
```

### Output:

```
* Serving Flask app 'lab15'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with watchdog (windowsapi)
* Debugger is active!
* Debugger PIN: 615-148-475
```

### Explanation:

This API manages warehouse items by storing item name, ID, and quantity. It supports CRUD operations and includes validation to prevent negative stock values. The system ensures proper tracking of inventory levels.

### Task 3 – Hotel Room Booking API

#### Task:

Develop a RESTful API for hotel room booking.

#### Instructions:

- Endpoints required:
  - o GET /rooms → View available rooms
  - o POST /rooms/book → Book a room
  - o GET /rooms/{id} → View booking details
  - o DELETE /rooms/{id} → Cancel booking
- Implement error handling for unavailable rooms.

#### Expected Output:

- API simulating hotel room booking operations.

#### PROMPT:

"""Generate a Flask REST API for hotel room booking using in-memory data.

Endpoints: GET /rooms, POST /rooms/book, GET /rooms/<id>, DELETE /rooms/<id>.

Add error handling if a room is already booked."""

#### Code:

```
1 """Generate a Flask REST API for hotel room booking using in-memory data.
2 Endpoints: GET /rooms, POST /rooms/book, GET /rooms/<id>, DELETE /rooms/<id>.
3 Add error handling if a room is already booked."""
4 from flask import Flask, request, jsonify
5 app = Flask(__name__)
6 # In-memory storage for rooms
7 rooms = {
8     1: {"type": "single", "price": 100, "booked": False},
9     2: {"type": "double", "price": 150, "booked": False},
10    3: {"type": "suite", "price": 300, "booked": False}
11 }
12 @app.route('/rooms', methods=['GET'])
13 def get_rooms():
14     """Endpoint to get the list of all rooms with their details."""
15     return jsonify(rooms)
16 @app.route('/rooms/book', methods=['POST'])
17 def book_room():
18     """Endpoint to book a room. Expects JSON with 'id' of the room to be booked."""
19     data = request.get_json()
20     room_id = data.get('id')
21     if room_id not in rooms:
22         return jsonify({"error": "Room not found"}), 404
23     if rooms[room_id]['booked']:
24         return jsonify({"error": "Room already booked"}), 400
25     rooms[room_id]['booked'] = True
26     return jsonify({"message": f"Room {room_id} booked successfully"})
27 @app.route('/rooms/<int:room_id>', methods=['GET'])
28 def get_room(room_id):
29     """Endpoint to get details of a specific room by its ID."""
30     if room_id not in rooms:
31         return jsonify({"error": "Room not found"}), 404
32     return jsonify(rooms[room_id])
33 @app.route('/rooms/<int:room_id>', methods=['DELETE'])
34 def delete_room(room_id):
35     """Endpoint to delete a room by its ID. This will remove the room from the in-memory storage."""
36     if room_id not in rooms:
37         return jsonify({"error": "Room not found"}), 404
38     del rooms[room_id]
39     return jsonify({"message": f"Room {room_id} deleted successfully"})
40 if __name__ == '__main__': app.run(debug=True)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS POSTMAN CONSOLE

```
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with watchdog (windowsapi)
* Debugger is active!
* Debugger PIN: 615-148-475
```

## Output:

```
* Restarting with watchdog (windowsapi)
* Debugger is active!
* Debugger PIN: 615-148-475
```

## Explanation:

This task builds an API to simulate hotel room booking operations. Users can view available rooms, book rooms, check booking details, and cancel bookings. It also includes error handling to prevent double booking of rooms.

## Task 4 – Online Voting API

### Task:

Create a RESTful API for an online voting system.

### Instructions:

- Endpoints required:
  - o GET /candidates → List candidates
  - o POST /vote → Cast a vote
  - o GET /results → View voting results
- Ensure one vote per user using in-memory validation.

### Expected Output:

- API simulating a secure online voting process.

### PROMPT:

"""Create a Flask REST API for an online voting system.

Endpoints: GET /candidates, POST /vote, GET /results.

Ensure each user can vote only once using in-memory validation."""

### Code:

```
lab15py >...
1  """Create a Flask REST API for an online voting system.
2  Endpoints: GET /candidates, POST /vote, GET /results.
3  Ensure each user can vote only once using in-memory validation."""
4  from flask import Flask, request, jsonify
5  app = Flask(__name__)
6  # In-memory storage for candidates and votes
7  candidates = ["Alice", "Bob", "Charlie"]
8  votes = {}
9  @app.route('/candidates', methods=['GET'])
10 def get_candidates():
11     """Endpoint to retrieve the list of candidates."""
12     return jsonify(candidates)
13 @app.route('/vote', methods=['POST'])
14 def cast_vote():
15     """Endpoint to cast a vote for a candidate."""
16     data = request.get_json()
17     user_id = data.get('user_id')
18     candidate = data.get('candidate')
19     # Validate input
20     if not user_id or not candidate:
21         return jsonify({"error": "User ID and candidate are required."}), 400
22     if candidate not in candidates:
23         return jsonify({"error": "Invalid candidate."}), 400
24     if user_id in votes:
25         return jsonify({"error": "User has already voted."}), 400
26     # Cast the vote
27     votes[user_id] = candidate
28     return jsonify({"message": "Vote cast successfully."}), 200
29 @app.route('/results', methods=['GET'])
30 def get_results():
31     """Endpoint to retrieve the voting results."""
32     results = {candidate: 0 for candidate in candidates}
33     for vote in votes.values():
34         if vote in results:
35             results[vote] += 1
36     return jsonify(results)
37 if __name__ == '__main__':
38     app.run(debug=True)
39 # TESTING
40 # To test the API, you can use tools like Postman or curl to send requests to the endpoints:
41 # 1. Get candidates: curl http://localhost:5000/candidates
42 # 2. Cast a vote: curl -X POST http://localhost:5000/vote -H "Content-Type: application/json" -d '{"user_id": "user1", "candidate": "Alice"}'
43 # 3. Get results: curl http://localhost:5000/results

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GIT LENS  POSTMAN CONSOLE
Press CTRL+C to quit
* Restarting with watchdog (windowsapi)
* Debugger is active!
* Debugger PIN: 615-148-475
```

**Output:**

```
* Restarting with watchdog (windowsapi)
* Debugger is active!
* Debugger PIN: 615-148-475
```

**Explanation:**

This API allows users to view candidates, cast votes, and check results. It ensures one vote per user using in-memory validation, making the system fair and secure.

**Task 5 – Teacher Management API****Task:**

**Develop a RESTful API to manage teacher records.**

**Instructions:**

- **Endpoints required:**
  - o GET /teachers → List all teachers
  - o POST /teachers → Add a new teacher
  - o PUT /teachers/{id} → Update teacher details
  - o DELETE /teachers/{id} → Remove a teacher
- **Ensure proper error handling and JSON responses.**

**Expected Output:**

- **Functional API for managing teacher information.**

**PROMPT:**

"""Generate a Flask REST API to manage teachers with id, name, and subject.

Implement GET, POST, PUT, DELETE endpoints for /teachers.

Return JSON responses and handle errors properly."""

**Code:**

```
1 """Generate a Flask REST API to manage teachers with id, name, and subject.
2 Implement GET, POST, PUT, DELETE endpoints for /teachers.
3 Return JSON responses and handle errors properly."""
4 from flask import Flask, request, jsonify
5 app = Flask(__name__)
6 # In-memory storage for teachers
7 teachers = {}
8 @app.route('/teachers', methods=['GET'])
9 def get_teachers():
10     """Endpoint to retrieve all teachers."""
11     return jsonify(list(teachers.values())), 200
12 @app.route('/teachers', methods=['POST'])
13 def add_teacher():
14     """Endpoint to add a new teacher."""
15     data = request.get_json()
16     if not data or 'name' not in data or 'subject' not in data:
17         return jsonify({'error': 'Invalid input'}), 400
18     teacher_id = len(teachers) + 1
19     teacher = {
20         'id': teacher_id,
21         'name': data['name'],
22         'subject': data['subject']
23     }
24     teachers[teacher_id] = teacher
25     return jsonify(teacher), 201
26 @app.route('/teachers/<int:teacher_id>', methods=['PUT'])
27 def update_teacher(teacher_id):
28     """Endpoint to update an existing teacher."""
29     if teacher_id not in teachers:
30         return jsonify({'error': 'Teacher not found'}), 404
31     data = request.get_json()
32     if not data or 'name' not in data or 'subject' not in data:
33         return jsonify({'error': 'Invalid input'}), 400
34     teacher = teachers[teacher_id]
35     teacher['name'] = data['name']
36     teacher['subject'] = data['subject']
37     return jsonify(teacher), 200
38 @app.route('/teachers/<int:teacher_id>', methods=['DELETE'])
39 def delete_teacher(teacher_id):
40     """Endpoint to delete a teacher."""
41     if teacher_id not in teachers:
42         return jsonify({'error': 'Teacher not found'}), 404
43     del teachers[teacher_id]
44     return jsonify({'message': 'Teacher deleted'}), 200
45
46 # Press CTRL+C to quit
47 Restarting with watchdog (windowsapi)
48 Debugger is active!
49 Debugger PIN: 615-148-475
```

## Output:

```
Restarting with watchdog (windowsapi)
Debugger is active!
Debugger PIN: 615-148-475
```

**Explanation:** This task creates an API to manage teacher records with ID, name, and subject. It supports full CRUD operations and includes error handling with JSON responses for smooth data management.